

ساختار کلی پروژه

۱. ماژول‌های اصلی پروژه

- **Board (صفحه بازی):** نمایش وضعیت کنونی بازی از جمله موقعیت مهره‌ها و خانه‌های مسدود.
- **CustomPlayer (عامل هوشمند):** این کلاس مسئولیت تصمیم‌گیری در مورد حرکت بهینه را دارد.
- **توابع ارزیابی و کمکی:** توابعی برای ارزیابی موقعیت بازی، تشخیص تله‌ها، و بررسی وضعیت صفحه.

۲. الگوریتم‌های مورد استفاده

- الگوریتم **Alpha-Beta Pruning** برای بهینه‌سازی جستجوی درخت تصمیم‌گیری.
- ارزیابی وضعیت بازی با استفاده از **Heuristic Functions**.
- تنظیم عمق جستجو به صورت پویا بر اساس مراحل مختلف بازی.

الگوریتم‌ها

۱. الگوریتم Alpha-Beta Pruning

الگوریتم **آلفا-بتا** یک بهینه‌سازی برای جستجوی **Minimax** است که شاخه‌های غیرضروری را هرس کرده و سرعت جستجو را افزایش می‌دهد. این الگوریتم در کلاس **CustomPlayer** برای بررسی حرکات و انتخاب بهترین حرکت استفاده می‌شود.

پیاده‌سازی کلی

- اگر به عمق صفر رسیده باشیم، مقدار تابع ارزیابی برگردانده می‌شود.
- در صورتی که نوبت بازیکن ما باشد (**maximizing_player**): مقدار بیشینه‌ای از بین حرکات مجاز محاسبه شده و آلفا به‌روز می‌شود.
- در صورتی که نوبت حریف باشد (**minimizing_player**):

- مقدار کمینه‌ای از بین حرکات مجاز محاسبه شده و بتا بهروز می‌شود.
- اگر مقدار آلفا از بتا بیشتر شود، شاخه‌های بعدی بررسی نمی‌شوند (هرس اتفاق می‌افتد).

۲. تنظیم پویای عمق جستجو

با توجه به تعداد حرکات انجام‌شده، عمق جستجو تغییر می‌کند:

- حرکات اولیه (کمتر از ۱۰ حرکت): عمق جستجو کم (۲).
- حرکات میانی (۱۰ تا ۲۰ حرکت): افزایش عمق (۴).
- حرکات بعدی (۲۰ تا ۳۰ حرکت): افزایش عمق (۶).
- در حرکات پایانی، عمق جستجو باز هم افزایش می‌یابد.

توابع اصلی

1. توابع ارزیابی بازی

تابع *score در CustomEvalFn*

این تابع میزان ارزش یک وضعیت بازی را بر اساس:

- تعداد حرکات ممکن برای بازیکن
- تعداد حرکات ممکن برای حریف
- نزدیکی به مرکز بازی (مزیت استراتژیک)
- امکان فشار دادن مهره‌ی حریف به گوشه‌ها ارزیابی کرده و یک مقدار عددی را برمی‌گرداند.

2. توابع کمکی

تابع *detect_trap (تشخیص تله)*

این تابع بررسی می‌کند که آیا یک موقعیت خاص به‌عنوان تله در نظر گرفته می‌شود یا خیر. اگر مهره‌ای از شش جهت محاصره شده باشد، آن مهره در وضعیت خطرناک است.

3. عامل هوشمند CustomPlayer

عامل هوشمند ما با استفاده از ترکیبی از استراتژی‌های مختلف تصمیم‌گیری می‌کند:

- در اولین حرکت، یکی از حرکات تصادفی انتخاب می‌شود.
- سپس حرکات ناامن حذف می‌شوند.
- الگوریتم **Alpha-Beta Pruning** اجرا می‌شود تا بهترین حرکت ممکن انتخاب شود.
- اگر زمان کافی نباشد، یک حرکت تصادفی انجام می‌شود.

موارد خلاقانه :

بعد از تست کردن ایجنت برای جلوگیری از این نکته که هر کس اول شروع کند برنده است سعی شده که بازی از روند عادی خارج شود. لذا شروع حرکت با یک حرکت رندوم شروع می‌شود.
ولی نه هر حرکتی بلکه یکی از خانه ها $(1,1)$, $(1,5)$, $(5,1)$, $(5,5)$.

مورد دوم این است که برای اینکه در ابتدای بازی حرکات خیلی مهم نیستند و دسترسی بیشتری به خانه ها داریم برای اینکه تایم لیمیت نخوریم از عمق جستجو دینامیک استفاده می‌شود.

بازی به ۴ قسمت تقسیم شده و هر قسمت دو تا به عمق اضافه می‌شود و از عمق ۲ شروع و تا عمق ۸ می‌رود.

مورد سوم این است که قبل این که بریم سراغ الفابتا حرکات هامون و فیلتر می‌کنیم.

اینگونه که اول حرکاتی که در حاشیه هستند و در حرکات حریف هم هستند فیلتر می‌شوند.

نتایج جلوی رندوم:

```
File Edit View Terminal Tabs Help
move chosen: (3, 6, False)
 0 | 1 | 2 | 3 | 4 | 5 | 6 |
0 | Q2 | > < | > < | > < | > < |
1 | | > < | > < | > < | > < |
2 | > < | > < | > < | > < |
3 | | | > < | > < | > < | Q1 |
4 | | | > < | > < | > < |
5 | | | | > < | > < |
6 | | > < | | | > < |

RandomPlayer - Q2 Turn
move chosen: (1, 0, 0, False)
 0 | 1 | 2 | 3 | 4 | 5 | 6 |
0 | > < | > < | > < | > < | > < |
1 | Q2 | > < | > < | > < | > < |
2 | > < | > < | > < | > < |
3 | | | > < | > < | > < | Q1 |
4 | | | > < | > < | > < |
5 | | | | > < | > < |
6 | | | > < | | | > < |

CustomPlayer - Q1 Turn
move chosen: (6, 6, False)
 0 | 1 | 2 | 3 | 4 | 5 | 6 |
0 | > < | > < | > < | > < | > < |
1 | Q2 | > < | > < | > < | > < |
2 | > < | > < | > < | > < |
3 | | | > < | > < | > < |
4 | | | > < | > < | > < |
5 | | | | > < | > < |
6 | | | > < | | | Q1 |

RandomPlayer - Q2 Turn
CustomPlayer - Q1 has won. Reason: RandomPlayer - Q2 has no legal moves left.

agent player win : 99
random player win : 1

agent win rate : 0.99 %
~/Desktop/AIpro/AI-Project-Fall24 main 13 74 1m 0s
```

```
File Edit View Terminal Tabs Help
move chosen: (5, 2, False)
 0 | 1 | 2 | 3 | 4 | 5 | 6 |
0 | > < | > < | > < | > < | > < |
1 | > < | > < | > < | > < | Q1 |
2 | > < | > < | | > < | > < |
3 | | | | > < | > < | > < |
4 | | | | > < | > < | > < |
5 | | | Q2 | > < | > < |
6 | | | | | > < | > < |

RandomPlayer - Q1 Turn
move chosen: (0, 6, False)
 0 | 1 | 2 | 3 | 4 | 5 | 6 |
0 | > < | > < | > < | > < | Q1 |
1 | > < | > < | > < | > < | > < |
2 | > < | > < | | > < | > < |
3 | | | | > < | > < | > < |
4 | | | | > < | > < | > < |
5 | | | Q2 | > < | > < |
6 | | | | | > < | > < |

CustomPlayer - Q2 Turn
move chosen: (5, 1, False)
 0 | 1 | 2 | 3 | 4 | 5 | 6 |
0 | > < | > < | > < | > < | Q1 |
1 | > < | > < | > < | > < | > < |
2 | > < | > < | | > < | > < |
3 | | | | > < | > < | > < |
4 | | | | > < | > < | > < |
5 | | | Q2 | > < | > < |
6 | | | | | > < | > < |

RandomPlayer - Q1 Turn
CustomPlayer - Q2 has won. Reason: RandomPlayer - Q1 has no legal moves left.

random player win : 0
agent player win : 100

agent win rate : 1.0 %
~/Desktop/AIpro/AI-Project-Fall24 main 13 74 44s
```

